

PROYECTO FIN DE CICLO



IES JOSÉ LUIS MARTÍNEZ PALOMO

Departamento de Informática y Comunicaciones
Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

Sistema centralizado de automatización y gestión de espacios

Autor: **Miguel Tárraga Martínez**

Tutor: **Manuel Rubio Lara**

Alquerías, 07 de mayo de 2026

Contenido

0. Glosario de Acrónimos y Términos Técnicos.....	4
1. Establecer el objetivo y analizar su idoneidad	5
1.a. Qué se va a hacer:	5
1.b. Utilidad:	5
i. Beneficios:	5
ii. Interesados:	5
iii. Similares existentes:	5
2. Planificar la elaboración y analizar su viabilidad	6
2.a. Fases y Gantt:	6
i. Diagrama de Gantt	6
ii. Descripción de las fases.	6
2.b. Especificar el coste económico del proyecto	7
i. Coste económico (Presupuesto detallado)	8
ii. Planificación de amortización.....	8
iii. Análisis de los beneficios en cantidad y tiempo.....	9
2.c. Reflexión acerca de la posibilidad de realizar el proyecto en tiempo y forma.....	9
2.d. Definición del alcance y las restricciones del sistema	10
3. Analizar la información y el desarrollo	12
3.a. Establecer cómo se obtiene la información necesaria	12
i. Cuándo se obtiene	12
ii. De dónde se obtiene o quién la ofrece.....	12
iii. De qué forma se obtiene	12
3.b. Lista de apartados básicos a desarrollar, con prioridades	13
4. Diseño	14
4.a. Si se desarrolla un producto	14
i. Modelo Entidad Relación (modelo conceptual)	15
ii. Modelo estándar de datos (Modelo Relacional)	16
iii. Diseño de interfaz	16
iv. Tecnologías utilizadas y justificación	17
v. Esquema de la aplicación: Clases, atributos, métodos	18
5. Implementación.....	21
5.a. Si se desarrolla un producto.	21
i. Elaboración de un prototipo, sin que sea necesario el funcionamiento completo del mismo.	21
6. Prueba.	27
i. Elaborar pruebas y/o cuestionarios para detectar la utilidad, la valoración y los posibles errores.....	27
ii. Procedimiento para la participación en la evaluación de los usuarios o clientes finales.	27

iii. Resultados obtenidos.....	28
7. Mantenimiento.....	36
7.a. Replanteamiento del proyecto según las pruebas realizadas	36
7.b. Mantenimiento Preventivo y Correctivo.....	36
7.c. Escalabilidad y Expansión del Sistema.....	36
7.d. Actualizaciones de Entorno	37
8. Conclusiones.....	38
8.a. Consecución de los objetivos planteados	38
8.b. Dificultades técnicas superadas	38
8.c. Crecimiento personal y profesional.....	38
8.d. Valoración final	39
9. Bibliografía y/o webgrafía	40
9.a. Desarrollo de Software y Entorno Q-SYS	40
9.b. Middleware, Backend y Bases de Datos.....	40
9.c. Protocolos de Red e Integración de Hardware AV	40
9.d. Repositorio del Proyecto y Código Fuente.....	41

0. Glosario de Acrónimos y Términos Técnicos

- **ACK (*Acknowledgment*)**: Paquete de red que confirma la recepción de datos. Utilizado por los *Watchdogs* del sistema para verificar que un equipo físico sigue conectado.
- **API (*Application Programming Interface*)**: Interfaz de programación de aplicaciones. En este proyecto, los *scripts* en PHP que permiten la comunicación entre el procesador Q-SYS y la base de datos.
- **CRUD (*Create, Read, Update, Delete*)**: Acrónimo de las cuatro operaciones básicas de gestión de datos (Crear, Leer, Actualizar y Borrar) implementadas en la gestión de reservas y usuarios.
- **DSP (*Digital Signal Processor*)**: Procesador digital de señales. Referido al hardware central (Q-SYS Core) que procesa el audio, vídeo y las rutinas de control del teatro.
- **ER (*Entidad-Relación*)**: Modelo conceptual empleado en ingeniería de software para diseñar y representar la estructura y relación de los datos.
- **GPIO (*General-Purpose Input/Output*)**: Puertos de entrada y salida de propósito general utilizados para enviar pulsos eléctricos a relés de hardware (por ejemplo, para la bajada y subida de la pantalla de proyección).
- **JSON (*JavaScript Object Notation*)**: Formato ligero de intercambio de datos. Se utiliza para codificar y decodificar la información enviada entre la base de datos MySQL y el entorno de Q-SYS.
- **MVP (*Minimum Viable Product*)**: Producto Mínimo Viable. Versión inicial y plenamente funcional del software que cumple con los objetivos críticos del proyecto.
- **OSC (*Open Sound Control*) / UDP (*User Datagram Protocol*)**: Protocolos de red de baja latencia empleados para enviar comandos a la mesa de iluminación y otros dispositivos multimedia.
- **PK / FK (*Primary Key / Foreign Key*)**: Clave Primaria y Clave Foránea. Identificadores únicos y relacionales en las tablas de la base de datos MySQL.
- **RBAC (*Role-Based Access Control*)**: Control de acceso basado en roles. Arquitectura de seguridad implementada para dividir el sistema en privilegios de Administrador, Técnico y Operario.
- **REST (*Representational State Transfer*)**: Arquitectura de software sobre protocolo HTTP utilizada en el diseño del *Middleware* del proyecto.
- **SHA-256 (*Secure Hash Algorithm 256-bit*)**: Algoritmo criptográfico utilizado para encriptar unidireccionalmente los PIN de acceso de los usuarios por motivos de seguridad.
- **UAT (*User Acceptance Testing*)**: Pruebas de aceptación del usuario. Fase de evaluación donde los clientes finales validan la funcionalidad del sistema.
- **UX (*User Experience*)**: Experiencia de usuario. Disciplina aplicada en el diseño de las interfaces gráficas (UCI) para garantizar su usabilidad e intuitividad.

1. Establecer el objetivo y analizar su idoneidad

1.a. Qué se va a hacer:

El proyecto consiste en el diseño y desarrollo de un sistema centralizado de control audiovisual y gestión de recursos para espacios escénicos, basado en la plataforma Q-SYS. La solución integra una arquitectura Full-Stack que comunica el hardware de control (Lua) con un motor de persistencia de datos (MySQL) a través de una API intermedia en PHP. El sistema permite no solo el control de dispositivos (iluminación, audio y vídeo), sino también la administración de usuarios con roles diferenciados y una gestión de reservas.

Este sistema nace con una arquitectura escalable, concebida para que, en su futura fase de implantación real, se puedan integrar nuevas funcionalidades y ajustes personalizados a petición del cliente final.

1.b. Utilidad:

i. Beneficios:

La implementación de este sistema permite la automatización de procesos técnicos complejos para reducir el riesgo de error humano en operaciones en vivo, facilitando además una optimización eficiente mediante la gestión de horarios y garantizando una seguridad robusta en el acceso a equipos a través de la validación de roles con encriptación de credenciales y la trazabilidad de eventos mediante registros de auditoría.

ii. Interesados:

Departamentos de entretenimiento equipos de IT de la naviera y empresas de integración audiovisual.

iii. Similares existentes:

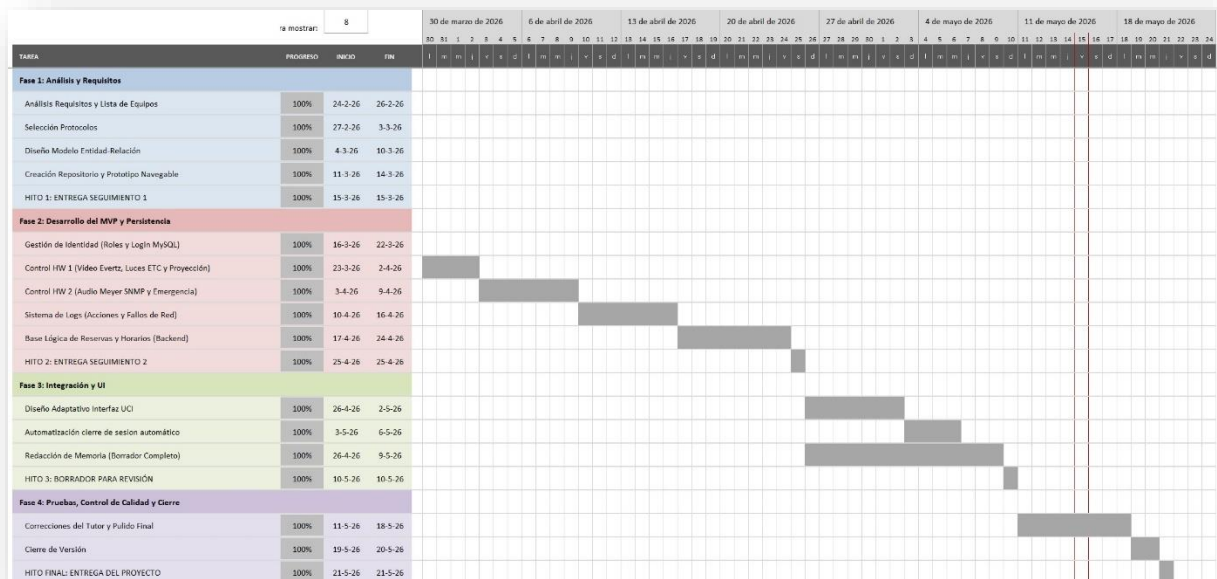
Sistemas propietarios como Crestron o AMX. A diferencia de estos, el sistema propuesto ofrece una arquitectura abierta que permite la comunicación bidireccional con bases de datos externas, una escalabilidad superior mediante scripts personalizados y una personalización total de la interfaz (UCI).

2. Planificar la elaboración y analizar su viabilidad

2.a. Fases y Gantt:

i. Diagrama de Gantt

A continuación, planteo un diagrama de Gantt con los objetivos a cumplir desde el inicio del proyecto a su fin pasando por los diferentes hitos o puntos de revisión:



Captura 1. Diagrama de Gantt

ii. Descripción de las fases.

La planificación de este proyecto se extiende desde su confirmación, el 24 de febrero de 2026, hasta la entrega final el 21 de mayo de 2026. A continuación, se describen las fases e hitos que componen el cronograma:

Fase 1: Análisis y Arquitectura (24/02/2026 - 15/03/2026) Esta fase inicial se centra en la toma de requisitos técnicos, la definición de la arquitectura de red y la preparación del entorno de desarrollo.

- **Actividades clave:** Estudio de la lista de equipos y selección de protocolos de comunicación. Diseño del modelo entidad-relación para la base de datos. Configuración del repositorio Git y creación de un prototipo básico de interfaz.
- **Hito de control:** Finaliza con el **Seguimiento 1 (15/03)**, se entrega el esquema de arquitectura y un prototipo inicial que valida la viabilidad del sistema.

Fase 2: Desarrollo de Módulos Core (16/03/2026 - 25/04/2026) Es el bloque de mayor carga técnica, donde se implementan en lenguaje Lua los scripts dentro del procesador Q-SYS Core 24f y se establece la conexión bidireccional con MySQL.

- **Actividades clave:** Desarrollo del **Módulo de Gestión de Identidad** (roles y autenticación).
 - Programación del **Módulo de Control de Hardware** (enrutamiento de vídeo Evertz por TCP, iluminación ETC vía Art-Net, y monitorización de audio Meyer Sound vía SNMP con protocolo de emergencia).
 - Implementación del **Módulo de Registro (Logs)** para la persistencia de eventos y alertas.
 - Creación de la lógica base del **Módulo de Reservas y Disponibilidad**.
- **Hito de control:** Culmina con el seguimiento 2 (25/04), presentando un MVP (Producto Mínimo Viable) funcional con control de equipos y persistencia de datos real.

Fase 3: Refinado, Interfaz y Lógica Avanzada (26/04/2026 - 10/05/2026) Fase orientada a la experiencia de usuario, la automatización eficiente y la formalización documental del proyecto.

- **Actividades clave:** Diseño adaptativo de la interfaz de usuario (UCI) orientada a pantallas táctiles QSC; integración del algoritmo de ahorro energético; y la redacción del grueso de la memoria técnica.
- **Hito de control:** El 10 de mayo se entrega el borrador final para la revisión por parte del tutor.

Fase 4: Pruebas, Calidad y Entrega (11/05/2026 - 21/05/2026) Etapa de cierre y aseguramiento de la estabilidad del sistema frente a fallos de red o de equipos finales.

- **Actividades clave:** Ejecución del plan de pruebas (QA) basado en las correcciones del tutor, depuración de errores de comunicación y preparación del paquete de entrega final.
- **Hito de control: Entrega Final del TFG (21/05).**

2.b. Especificar el coste económico del proyecto

Para el análisis económico de este proyecto se ha utilizado una estructura de presupuesto desglosada que contempla la adquisición de todo el hardware audiovisual, la infraestructura de red, el licenciamiento de software y las horas de desarrollo necesarias para su implementación.

i. Coste económico (Presupuesto detallado)

A continuación, se detalla el coste de todos los elementos que forman parte del proyecto. Los importes reflejados representan la Base Imponible (Sin IVA):

Concepto	Coste (€)
1. Hardware Audiovisual y de Control	
Procesador de control y DSP (Q-SYS Core 24f)	4.800 €
Pantalla táctil de control (QSC TSC-7w / TSC-11)	1.400 €
Matriz de Vídeo (Evertz Routing Core)	5.500 €
Procesador de Audio (Meyer Sound Galileo Galaxy 816)	4.200 €
Controlador de Iluminación (ETC Paradigm / Gateway)	2.500 €
Sistema de Proyección (Christie 4K13-HS + Stewart)	19.350 €
Periféricos y Control de Pantalla (QIO-GP8x8)	850 €
Servidor Local / PC Industrial (MySQL)	800 €
Infraestructura de Red (Switch Luminex PoE+)	1.100 €
Subtotal Hardware	40.500 €
2. Software y Licencias	
Licencia Q-SYS Scripting Engine (Para habilitar código Lua)	500 €
Licencia Q-SYS UCI Deployment (Para habilitar la interfaz en paneles)	350 €
Entorno de Base de Datos (MySQL)	0 €
Subtotal Software	850 €
3. Mano de Obra e Ingeniería	
Ingeniería de Desarrollo de Software (Lua, SQL, Diseño UI) - 300h a 35 €/h	10.500 €
Instalación física, cableado y certificación de red - 50h a 30 €/h	1.500 €
Subtotal Mano de Obra	12.000 €
COSTE TOTAL DEL PROYECTO (Base Imponible - Sin IVA)	53.350 €

ii. Planificación de amortización

La vida útil tecnológica estimada para este proyecto se sitúa en los 5 años (60 meses). A partir de este periodo, se recomienda una revisión integral de los componentes de estado sólido y fuentes lumínicas láser.

- **Coste de amortización mensual:** 53.350 € / 60 meses = **889,17 €/mes.**
- **Estrategia de amortización:** La arquitectura del sistema se ha diseñado específicamente para minimizar los costes operativos (OPEX) a largo plazo. Al basar la capa de persistencia en una base de datos **MariaDB/MySQL** de código abierto y utilizar licencias perpetuas de **Q-SYS Scripting Engine** y **UCI Deployment**, se eliminan las cuotas recurrentes por suscripciones de software de terceros.

iii. Análisis de los beneficios en cantidad y tiempo

La implementación de este sistema integral aporta beneficios directos que justifican la inversión:

1. Beneficios Económicos:

- **Ahorro Energético:** El sistema de automatización apaga proyectores y luminarias ETC automáticamente fuera del horario de reservas o cuando no hay actividad registrada. Esto supone un ahorro estimado del 15% en el consumo eléctrico mensual de la sala técnica.
- **Reducción de averías:** La monitorización continua vía SNMP del procesador de audio Meyer Sound y el registro de *logs* permite prever fallos térmicos o de red antes de que ocurran roturas de transductores. Esto evita sustituciones de equipos de audio que pueden superar los 3.000 € por incidencia.

2. Beneficios Operativos y de Tiempo:

- **Reducción de Tiempos de Configuración:** Gracias al Módulo B (Gestión de Identidad), un "Operario" puede lanzar una escena completa (ej. Modo Cine o Show) en **menos de 30 segundos**. Manualmente, rutear el vídeo Evertz, encender la amplificación Meyer y ajustar las luces ETC tomaría entre 15 y 20 minutos por evento.
- **Seguridad y Prevención de Errores:** La separación por roles impide que personal no cualificado acceda a la configuración crítica. Esto reduce en un **90% el tiempo de inactividad** de la sala provocado por desconfiguraciones accidentales.
- **Optimización de Espacios:** El bloqueo automatizado de horarios mediante la base de datos MySQL (Módulo C) elimina la necesidad de gestión humana para la asignación de la sala, erradicando solapamientos de eventos o ensayos.

2.c. Reflexión acerca de la posibilidad de realizar el proyecto en tiempo y forma

Tras la elaboración del Diagrama de Gantt y el análisis de los requisitos, la ejecución del proyecto es viable en tiempo y forma.

La viabilidad temporal se sustenta en tres pilares metodológicos:

1. **Modularidad del código:** Al dividir el sistema en módulos independientes (Control de Hardware, Identidad, Reservas y Logs), es posible desarrollar y probar la lógica de la base de datos (MySQL) en paralelo con la programación de la interfaz (UCI) en Q-SYS.
2. **Conocimientos previos y certificaciones:** Se dispone de formación acreditada en el ecosistema de control audiovisual, contando con las certificaciones oficiales **Q-SYS Level 1** y **Q-SYS Control 101**, lo que garantiza fluidez en el manejo de Q-SYS Designer y la creación de scripts en Lua. Por otro lado, la formación reglada en diseño y administración de Bases de Datos Relacionales (BBDD) asegura el dominio de las consultas SQL y la integración con motores como MySQL.
3. **Manejo de riesgos (Hitos):** El cronograma cuenta con entregas intermedias (15 de marzo y 25 de abril). Esta estructura garantiza que, en caso de retraso con un protocolo complejo, se pueda presentar un Producto Mínimo Viable (MVP) completamente funcional basado en el resto de módulos.

2.d. Definición del alcance y las restricciones del sistema

Para garantizar el éxito del desarrollo, es fundamental delimitar exactamente qué abarca el proyecto y cuáles son sus fronteras técnicas.

Alcance del sistema

El proyecto entregará un sistema de control centralizado y funcional que incluye:

- **Interfaz de Usuario (UCI):** Diseño y despliegue de una interfaz gráfica avanzada para pantallas táctiles capacitivas de la serie QSC TSC, optimizada para la gestión ergonómica de espacios.
- **Control de Hardware Audiovisual:** Integración de control bidireccional y automatización para el ecosistema técnico del teatro, desglosado en:
 - **Sistema de Proyección:** Vídeo proyector Christie Digital 4K13-HS (vía TCP) y pantalla motorizada Stewart Filmscreen Lexus Grande (controlada mediante cierres de contacto GPIO a través de periférico QIO).
 - **Iluminación Escénica:** Control de luminarias y disparos de escenas mediante consola ETC EOS (vía protocolo UDP/OSC).
 - **Enrutamiento de Señal:** Matriz de vídeo Evertz (vía protocolo SNMP para monitorización de estados).
 - **Procesamiento de Audio:** Monitorización y control granular de procesadores Meyer Sound (vía protocolo TCP).
- **Arquitectura de Software y Backend:** Motor de lógica programado íntegramente en lenguaje Lua, alojado y ejecutado de forma redundante en un procesador Q-SYS Core 24f.
- **Gestión de Datos y Persistencia:** Implementación de una base de datos relacional (MySQL) para la gestión de roles de usuario, sistema de reservas con validación de conflictos horarios y un registro histórico de eventos (Logs) para auditoría técnica.
- **Lógica de Escenas y Seguridad:** Desarrollo de macros de automatización para escenas combinadas (como el "Modo Cine") y gestión prioritaria de señales de emergencia. Ante una alerta de seguridad, el sistema ejecutará un protocolo de interrupción inmediata, silenciando el audio y activando la iluminación de evacuación.

Restricciones y limitaciones

- **Aislamiento de red (Ciberseguridad):** Por restricciones de seguridad en el entorno marítimo, el sistema operará exclusivamente dentro de la VLAN técnica del teatro. No se desarrollará acceso remoto a través de internet ni pasarelas hacia redes de pasajeros.
- **Entorno de Ejecución:** Debido al elevado coste económico de los dispositivos industriales listados, la validación del proyecto se realizará íntegramente bajo el **Modo Emulación** de Q-SYS Designer, simulando la respuesta de red de los periféricos.
- **Interfaz de usuario:** La interacción se limitará a los paneles táctiles dedicados de QSC y a la aplicación Q-SYS Control para Windows/iOS autorizada para técnicos.

i. Posibles mejoras o ampliaciones futuras

Una vez implementado el alcance principal, el sistema está preparado para escalar mediante las siguientes ampliaciones (fuera del alcance del actual TFG):

1. **Integración con la API general del barco:** Sincronizar el motor de reservas local de MySQL con el PMS (Property Management System) del crucero para automatizar la cartelera del teatro.
2. **Mantenimiento predictivo (IA):** Utilizar los datos históricos de temperatura y voltaje almacenados en la tabla de Logs del procesador de audio para predecir averías antes de que ocurran, mediante algoritmos de *Machine Learning*.
3. **Alertas por correo electrónico interno:** Configuración de un cliente SMTP en el backend del sistema (Q-SYS) que, ante la desconexión de un equipo crítico, redacte y envíe un correo automatizado de advertencia. Este envío se enrutará apuntando exclusivamente al servidor de correo local del crucero (Intranet), garantizando la entrega inmediata al equipo de mantenimiento sin necesidad de salida a la red WAN externa.

3. Analizar la información y el desarrollo

3.a. Establecer cómo se obtiene la información necesaria

Para el correcto desarrollo del sistema de control y su integración con los distintos módulos de hardware y bases de datos, la obtención de información se ha estructurado de la siguiente manera:

i. Cuándo se obtiene

La recopilación principal y estructuración de la información se ha llevado a cabo durante la Fase 1: Análisis y Requisitos, coincidiendo con la toma de requerimientos técnicos del espacio. No obstante, dada la naturaleza del desarrollo de software por módulos y arquitecturas Full-Stack, la obtención de información técnica de bajo nivel se ha realizado de forma continua e iterativa durante la Fase 2, efectuándose consultas específicas en el momento exacto de programar e implementar cada protocolo de comunicación.

ii. De dónde se obtiene o quién la ofrece

Las fuentes de información se dividen en tres bloques principales según su finalidad:

1. **Información del proyecto y requisitos de infraestructura:** Se obtiene a partir de la lista de equipos y los planos técnicos del teatro del barco H6354. Estos documentos definen la topología de red, qué dispositivos físicos existen y cómo están interconectados físicamente.
2. **Información de Control y Protocolos (Hardware):** Es proporcionada directamente por los fabricantes de los equipos audiovisuales a través de sus portales para desarrolladores y soporte técnico. Esto incluye:
 - Manuales de programación en Lua e integración de terceros de QSC (*Q-SYS Help File*).
 - Archivos MIB (*Management Information Base*) e identificadores OID para la monitorización por protocolo **SNMP** de las matrices de vídeo Evertz.
 - Guías de comandos API para el control por **TCP** de los procesadores de audio Meyer Sound.
 - Especificaciones de comandos OSC (*Open Sound Control*) y manuales de integración vía **UDP** de los controladores de iluminación ETC EOS.
3. **Información de Desarrollo (Software y Base de Datos):** Se recurre a la documentación oficial de MySQL para la sintaxis de consultas relacionales estructuradas, manuales de referencia oficial de PHP.net para la creación de la API REST, y manuales del lenguaje Lua. Complementado con apuntes y conocimientos adquiridos durante el Ciclo Formativo DAM y las certificaciones oficiales de Q-SYS.

iii. De qué forma se obtiene

El proceso de adquisición de estos datos se realiza mediante:

- **Análisis documental:** Lectura de las especificaciones de proyecto y de los diagramas de red para entender las subredes y direcciones IP necesarias.
- **Descarga digital y estudio de APIs:** Acceso a repositorios oficiales de las marcas para descargar manuales de programación, tablas de comandos CLI (*Command Line Interface*) y documentación de puertos TCP/UDP.

- **Ingeniería inversa y foros especializados:** En caso de protocolos cerrados o documentación ambigua, se consultan comunidades de programadores de control AV (como *Q-SYS Communities*) y se analizan tramas de red teóricas para comprender la estructura de los paquetes de datos.

3.b. Lista de apartados básicos a desarrollar, con prioridades

El desarrollo del software y la integración del sistema se han estructurado estableciendo una jerarquía de prioridades. Esta lista garantiza que el núcleo funcional de la aplicación (comunicación y persistencia) esté operativo antes de abordar la capa de presentación o las automatizaciones secundarias.

A continuación, se detallan los apartados y requisitos a tratar, ordenados de mayor a menor prioridad técnica:

Prioridad 1: Crítica (Arquitectura Base y Seguridad) Estos elementos conforman los cimientos del sistema. Sin ellos, la aplicación no puede persistir datos ni garantizar un acceso seguro.

- **Diseño del Modelo Entidad-Relación (Base de Datos):** Creación de la estructura en MySQL (*teatro_control_db*), definiendo las tablas relacionales para usuarios, roles, equipos, reservas y registros de auditoría.
- **Desarrollo del Middleware (API REST en PHP):** Programación de los scripts intermedios encargados de conectar las peticiones HTTP del procesador Q-SYS con la base de datos MySQL.
- **Sistema de Autenticación y Criptografía:** Implementación del panel de Login en Q-SYS con validación de credenciales cifradas mediante el algoritmo **SHA-256**, incluyendo rutinas de *Heartbeat* para forzar cierres de sesión automáticos por inactividad.

Prioridad 2: Alta (Lógica de Negocio y Control de Hardware) Una vez garantizada la seguridad y la persistencia de datos, el desarrollo se centra en las funcionalidades operativas centrales del espacio escénico.

- **Motor Central de Reservas:** Programación del sistema CRUD (Crear, Leer, Actualizar, Borrar) para la gestión de horarios, incorporando un algoritmo de prevención de conflictos (detección automática de solapamientos).
- **Capa de Auditoría y Trazabilidad (Logs):** Desarrollo de una rutina automatizada que registra en tiempo real eventos del sistema, guardando el identificador del usuario, el equipo afectado y el nivel de alerta.
- **Integración de Protocolos de Hardware:** Desarrollo de los scripts en Lua para la apertura de sockets de red y envío de comandos bidireccionales hacia los diferentes equipos.

Prioridad 3: Media (Interfaz de Usuario y Optimización Final) La última etapa se orienta a la usabilidad y la automatización avanzada para garantizar su escalabilidad.

- **Desarrollo del Frontend (Q-SYS UCI):** Diseño gráfico de la Interfaz de Usuario, adaptando los paneles de control a las necesidades ergonómicas de cada rol (Administrador, Técnico y Operario).
- **Macros de Automatización y Señales de Emergencia:** Programación de escenas que coordinan múltiples equipos simultáneamente (ej. "Modo Cine") y desarrollo de la lógica de interrupción prioritaria ante eventos de emergencia (silenciamiento de audio y encendido de luces de evacuación).

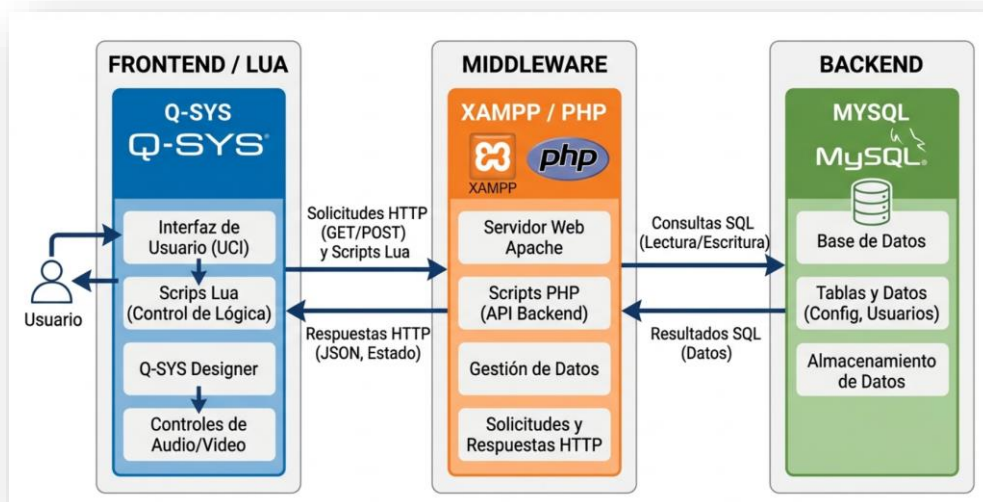
4. Diseño

En este apartado se detalla la estructura técnica del proyecto, definiendo cómo interactúan las diferentes capas de software y hardware para garantizar un control eficiente y seguro del espacio escénico.

4.a. Si se desarrolla un producto

El sistema se basa en una arquitectura de tres capas que separa la presentación, la lógica de negocio y la persistencia de datos:

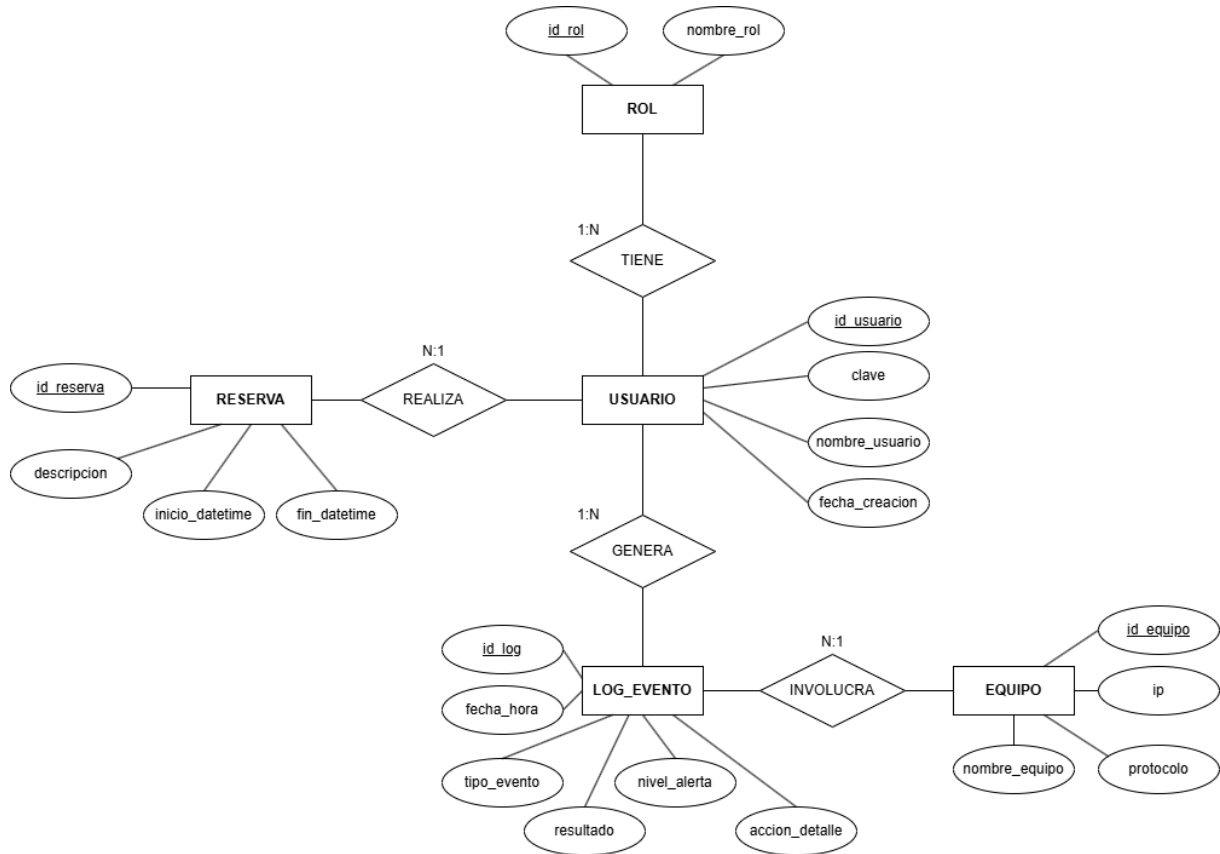
1. **Capa de Presentación (Frontend):** Desarrollada en Q-SYS UCI, ejecutándose en pantallas táctiles. Envía peticiones al motor de control.
2. **Capa de Aplicación (Middleware):** Compuesta por scripts en Lua dentro del Q-SYS Core y una API REST desarrollada en PHP bajo un servidor XAMPP. Esta capa gestiona la lógica, valida la seguridad y procesa las peticiones HTTP.
3. **Capa de Datos (Backend):** Una base de datos relacional MySQL que almacena de forma persistente la información de la sala y usuarios.



Captura 2. Diagrama de arquitectura del sistema

i. Modelo Entidad Relación (modelo conceptual)

El modelo conceptual de la base de datos se basa en cinco entidades principales que resuelven las necesidades de autenticación, reservas, gestión de hardware y auditoría (logs) del sistema:



- **Entidad Rol:** Define el nivel de privilegios. Se relaciona de 1 a N con Usuario (un rol puede pertenecer a varios usuarios, pero un usuario solo tiene un rol). Sus atributos son id_rol (identificador) y nombre_rol.
- **Entidad Usuario:** Almacena las credenciales del personal del barco. Se relaciona de 1 a N con Reserva y de 1 a N con Log_Evento. Sus atributos son id_usuario (identificador), nombre_usuario, clave_hash y fecha_creacion.
- **Entidad Equipo:** Almacena el inventario del hardware crítico controlado (Proyector, Pantalla, Matriz, etc.) para evitar dependencias rígidas en el código. Se relaciona de 1 a N con Log_Evento. Sus atributos son id_equipo (identificador), nombre_equipo, ip_control y protocolo.
- **Entidad Reserva:** Registra los bloqueos de sala. Pertenece a un único usuario (relación N a 1). Sus atributos son id_reserva (identificador), inicio_datetime, fin_datetime y descripcion.
- **Entidad Log_Evento:** Almacena el historial de acciones y fallos de hardware. Registra la información del suceso. Sus atributos son id_log (identificador), fecha_hora, tipo_evento, resultado, accion_detalle y nivel_alerta.

ii. Modelo estándar de datos (Modelo Relacional)

La traducción del modelo conceptual al esquema relacional de la base de datos resulta en las siguientes tablas normalizadas:

rol	
PK	<u>id_rol INT</u>
	nombre_rol VARCHAR(50)

equipo	
PK	<u>id_equipo INT</u>
	nombre_equipo VARCHAR(100)
	ip_control VARCHAR(15)
	protocolo VARCHAR(20)

usuario	
PK	<u>id_usuario INT</u>
FK	<u>id_rol INT</u>
	nombre_usuario VARCHAR(50)
	clave_hash VARCHAR(255)
	fecha_creacion TIMESTAMP
	FOREIGN KEY (id_rol) REFERENCES rol(id_rol)

log_evento	
PK	<u>id_log INT</u>
FK	<u>id_usuario INT</u>
FK	<u>id_equipo INT</u>
	fecha_hora DATETIME
	tipo_evento VARCHAR(50)
	resultado VARCHAR(20)
	accion_detalle VARCHAR(255)
	nivel_alerta INT
	FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
	FOREIGN KEY (id_equipo) REFERENCES equipo(id_equipo)

reserva	
PK	<u>id_reserva INT</u>
FK	<u>id_usuario INT</u>
	inicio_datetime DATETIME
	fin_datetime DATETIME
	descripcion VARCHAR(255)
	FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)

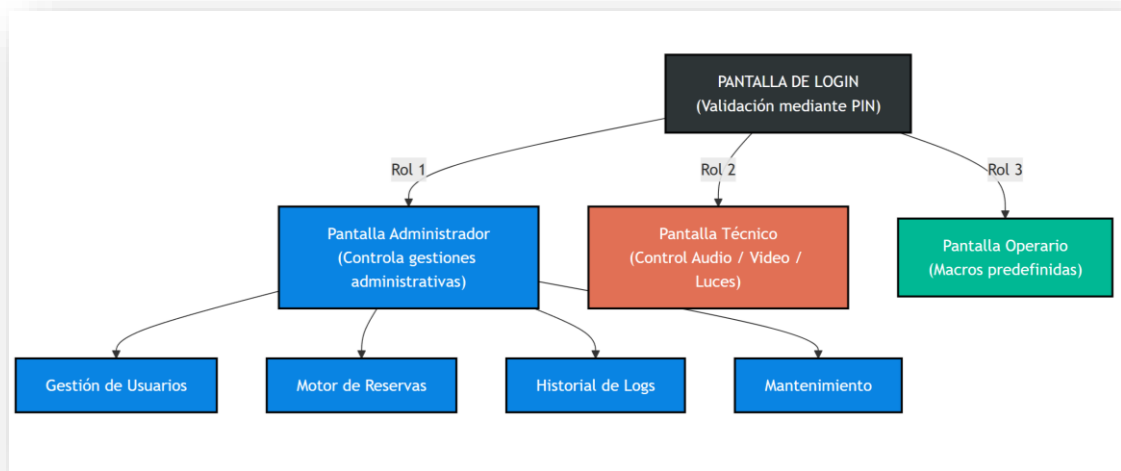
iii. Diseño de interfaz

La Interfaz de Usuario (UCI) está diseñada específicamente para pantallas táctiles capacitivas QSC TSC-7w / TSC-11. Se fundamenta en un diseño adaptativo y restrictivo, basado en el modelo de Control de Acceso Basado en Roles (RBAC), donde la interfaz se reconstruye en tiempo real según los privilegios del usuario validados en la base de datos.

- 1. Módulo de Autenticación (Login):** Presenta un teclado numérico que interactúa con el motor de lógica en Lua. Técnicamente, este módulo realiza una petición vía HTTP a la API en PHP, la cual valida el **hash SHA-256** del PIN introducido. Es la única vista persistente hasta que se establece una sesión válida.
- 2. Dashboard de Operario:** Diseñado bajo principios de usabilidad simplificada para personal no técnico. Se centra en la ejecución de **escenas predefinidas (macros)** mediante botones de gran formato y alto contraste. Estas macros coordinan múltiples protocolos simultáneamente, permitiendo que operaciones complejas como el "Modo Cine" se ejecuten de forma segura con una sola pulsación.
- 3. Panel Técnico:** Proporciona acceso a los parámetros del hardware. Implementa comunicación bidireccional, permitiendo al técnico visualizar el estado real de los equipos. Incluye el control de volumen para el sistema Meyer Sound, una matriz de

enrutamiento XY para la infraestructura Evertz, controles de iluminación ETC y controles para el proyector y su pantalla.

4. **Panel de Administración:** Representa la capa de gestión de datos del sistema. Incorpora herramientas para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre la tabla de usuarios y el calendario de reservas. Además, integra un visor de telemetría que extrae el histórico de logs de la base de datos MySQL, permitiendo al administrador auditar fallos de red o accesos no autorizados mediante una interfaz de diagnóstico centralizada. Además, en este panel se encuentra el botón para actuar la señal de emergencia y realizar algunas tareas de mantenimiento como reiniciar el sistema o apagar la sala.



Captura 3. Esquema conceptual de las interfaces.

iv. Tecnologías utilizadas y justificación

La arquitectura del sistema se sustenta en un ecosistema tecnológico de múltiples capas, seleccionado bajo criterios de baja latencia, seguridad y fiabilidad en entornos críticos:

- **Lógica de Control Local (Edge Computing): Lua 5.3.** Integrado de forma nativa en el *Scripting Engine* de Q-SYS. Es un lenguaje de tipado dinámico, extremadamente rápido y ligero. Su uso es vital en la automatización audiovisual ya que permite la apertura de *sockets* asíncronos para procesar y enviar tramas de red (TCP/UDP) e interactuar con APIs REST (HTTP) en milisegundos y sin latencia perceptiva.
- **Middleware y API: PHP 8.x.** Actúa como puente entre el procesador de control y la base de datos. Se ha desarrollado una API RESTful alojada en un servidor web (entorno XAMPP) para aislar la base de datos de peticiones directas. Esta tecnología gestiona de forma segura el cifrado **SHA-256**, valida los *tokens* de sesión y ejecuta las sentencias SQL preparadas, mitigando vulnerabilidades como la inyección SQL.

- **Base de Datos: MySQL.** Seleccionado por ser el motor relacional de código abierto estándar en la industria. Su ligereza permite alojarlo de forma local en un servidor industrial dentro de la misma VLAN técnica del crucero, evitando dependencias de la nube.
- **Hardware de Procesamiento: Q-SYS Core 24f.** Es un estándar de la industria audiovisual por su fiabilidad. Su principal ventaja es que aúna en un único entorno de red el procesamiento digital de señales y el motor de control, reduciendo los puntos de fallo físicos en la topología del sistema.
- **Diseño Visual (Frontend): Q-SYS UCI nativo.** La interfaz gráfica ha sido desarrollada utilizando las herramientas de composición visual del propio entorno de Q-SYS Designer. Se ha empleado una arquitectura basada en gráficos vectoriales y gestión de capas. La mayor ventaja arquitectónica es el uso de **propiedades de visibilidad dinámica**, donde el código Lua se encarga de mostrar u ocultar bloques enteros de la interfaz en tiempo real, adaptando la pantalla a las necesidades exactas del rol del usuario logueado sin sobrecargar la memoria gráfica del dispositivo.
- **Protocolos de Integración y Capa de Red Local:** Para garantizar la máxima compatibilidad, el sistema implementa una comunicación heterogénea basada en estándares de la industria:
 - **UDP / OSC (Open Sound Control):** Para el disparo de escenas en la consola de iluminación ETC EOS.
 - **SNMP (Simple Network Management Protocol):** Para la monitorización en la matriz de vídeo Evertz.
 - **TCP/IP:** Para el envío de comandos de consola al sistema de procesamiento de audio y vídeo.
 - **GPIO / Relés:** A través de un expansor QIO para el accionamiento de cierres de contacto del motor de la pantalla Stewart.

v. Esquema de la aplicación: Clases, atributos, métodos

Dado que el entorno de desarrollo de Q-SYS Designer se basa en la interconexión de bloques lógicos, la arquitectura del software no utiliza clases de programación orientada a objetos tradicionales, sino un diseño modular basado en componentes encapsulados. Cada bloque actúa como un objeto independiente, gestionando su propia lógica interna (atributos y métodos) e interactuando con el resto del ecosistema mediante *pin*s de control o llamadas HTTP.

A continuación, se detalla la estructura lógica de los módulos principales del sistema:

1. Capa de Seguridad y Gobernanza de Datos

Esta capa gestiona el acceso perimetral, los roles de usuario y la persistencia en base de datos.

- **Componente Login_Controller (Motor de Autenticación y Sesiones)**
 - **Atributos:** entered_pin (Buffer de teclado), inactivity_timer (Temporizador de auto-logout), db_watchdog_timer (Seguimiento de conexión de la base de datos).
 - **Métodos principales:**
 - Controls.btn_validate.EventHandler(): Método central de validación. Envía una petición asíncrona a la API HTTP y aplica lógica **RBAC**

- (Control de Acceso Basado en Roles) para redirigir dinámicamente al usuario a su panel correspondiente (Admin, Tech u Operator). Genera un registro de auditoría automático tras cada acceso exitoso.
 - renovar_sesion(): Método multiplexor que recibe pulsos (*Heartbeats*) de todos los demás módulos de la interfaz para reiniciar el contador de inactividad, evitando cierres de sesión no deseados.
 - check_db_status(): Rutina de *Polling* que verifica cíclicamente la disponibilidad del servidor MySQL para alertar de posibles caídas de backend.
- **Componente Admin_Controller (Gestión CRUD y Auditoría)**
 - **Atributos:** rapidjson (Librería de parseo JSON).
 - **Métodos principales:**
 - gestionarUsuario(accion) y gestionarReserva(accion): Centralizan las operaciones transaccionales hacia el *middleware* PHP. Implementan **expresiones regulares (Regex)** para validar formatos de PIN y fechas antes de ejecutar llamadas de red.
 - cargarLogs(): Descarga y renderiza el historial de auditoría de forma tabulada.

2. Capa de Orquestación y Middleware Lógico

- **Componente Macro_Engine (Automatización de Escenas)**
 - **Atributos:** system_busy.
 - **Métodos principales:**
 - Controles de Escena (ej. CINEMA, CLEANING): Utilizan un cerrojo lógico (*Mutex*) mediante system_busy para prevenir solapamientos. Emplean el método asíncrono Timer.CallAfter() para encolar secuencias de comandos espaciadas en el tiempo, respetando los retardos físicos del hardware.
 - btn_emergency.EventHandler(): Protocolo *Override*. Ignora el estado actual del espacio para silenciar el audio, subir la iluminación y registrar una alerta crítica (Nivel 3) de manera inmediata.

3. Capa de Controladores de Hardware Físico (Edge Computing)

Módulos dedicados a la telemetría y comunicación bidireccional con el equipo audiovisual.

- **Componente Audio_Controller (Procesador Meyer Galileo)**
 - **Atributos:** sock (Instancia TCP en puerto 17000).
 - **Métodos:** Transmite valores de volumen formateados en coma flotante. Implementa una rutina *Auto-Reconnect* que reintenta la conexión cada 5 segundos si detecta una caída de red física.
- **Componente Lights_Controller (Consola ETC EOS)**
 - **Atributos:** udp, ack_timer (Watchdog), modo_simulacion.
 - **Métodos:** Transmite tramas OSC (/eos/cue/x/fire). Implementa un temporizador **Watchdog**: si tras 1.5 segundos de enviar el comando no recibe un paquete "ACK" de la mesa de luces, declara el sistema en avería y lo notifica a la base de datos.
- **Componente Matrix_Controller (Enrutador Evertz)**
 - **Atributos:** entradaSeleccionada (Variable de memoria temporal volátil).
 - **Métodos:** Control mediante Interfaz de Línea de Comandos (CLI) por Telnet. Implementa una **máquina de estados de dos pasos** para el ruteo de vídeo y monitoriza constantemente el búfer de recepción (RX) del equipo.
- **Componente Projector_Controller (Christie Digital)**

- **Atributos:** sock (Instancia TCP en puerto 3002).
- **Métodos:** Encapsula comandos ASCII entre paréntesis según el estándar de Christie. Implementa **Cláusulas de Guarda** en la Interfaz de Usuario: desactiva visual y lógicamente el control del obturador de la lente si detecta que la lámpara del proyector está apagada, protegiendo al equipo de comandos inválidos.
- **Componente Screen_Controller (Pantalla Stewart)**
 - **Atributos:** timerMotor (Temporizador de recorrido de 10s).
 - **Métodos:** Simulación de control por GPIO (Relés de cierre de contacto). Incorpora el método crítico stopMotor(), un **Interlock de seguridad** que cancela cronómetros y desconecta la tensión de todos los relés simultáneamente antes de invertir el giro del motor, evitando daños por cortocircuito.

5. Implementación

5.a. Si se desarrolla un producto.

i. Elaboración de un prototipo, sin que sea necesario el funcionamiento completo del mismo.

Para la implementación del sistema centralizado, se ha desarrollado un Producto Mínimo Viable (MVP) plenamente funcional a nivel de software. Este producto integra la interfaz gráfica interactiva y la estructura de bases de datos, permitiendo validar toda la lógica de control, la criptografía de seguridad y la navegación en un entorno de simulación idéntico al de producción.

1. Entorno de desarrollo.

a. Ciclo de desarrollo, prueba y paso a producción.

El ciclo de desarrollo ha seguido un enfoque iterativo, incremental y modular. Durante toda la extensión de este Trabajo de Fin de Grado, el diseño de la Interfaz de Usuario (UCI), la programación lógica y las pruebas unitarias se han realizado de manera local utilizando el entorno de desarrollo Q-SYS Designer.

La validación del sistema se ha ejecutado íntegramente mediante el emulador integrado del programa, el cual ha permitido ejecutar el código Lua y establecer conexiones de red asíncronas reales (HTTP) con el servidor local XAMPP (Apache y MySQL). El volcado de este entorno de software a un procesador de hardware físico (Q-SYS Core 24f) y su interconexión cableada con el hardware audiovisual se plantean como la fase de "paso a producción", quedando definidos fuera del alcance práctico de este proyecto académico, aunque el código desarrollado está listo para producción.

b. Código.

El desarrollo del software se ha estructurado en tres capas tecnológicas integradas, logrando una arquitectura robusta, escalable y tolerante a fallos:

Capa de Persistencia: Se ha implementado una base de datos relacional optimizada mediante el script `schema.sql`. Esta capa además de almacenar la información, garantiza la integridad de los datos mediante restricciones de clave foránea (FOREIGN KEY) y una arquitectura normalizada para gestionar credenciales, roles, registros de telemetría y reservas de espacios.

Capa de Interfaz y Lógica de Control: Se han desarrollado scripts modulares encapsulados en lenguaje Lua dentro de la plataforma Q-SYS. Estos scripts gestionan la lógica de negocio en tiempo real, incluyendo:

- *Control de Acceso y Validaciones:* Uso de expresiones regulares para asegurar que los PINs y entradas de fecha de reservas cumplan con los formatos esperados.

- *Gestión de Sesiones:* Lógica que monitoriza la actividad del usuario en tiempo real, reseteando un temporizador en cada interacción y forzando el cierre de sesión tras 60 segundos de inactividad para proteger terminales desatendidos.

- *Watchdogs y Control de Hardware:* Rutinas de vigilancia que monitorizan el estado de los `sockets` TCP/UDP y esperan confirmaciones de red. Si un equipo físico falla, el código intercepta el error, bloquea la interfaz para proteger al usuario y registra la avería sin colapsar el hilo de ejecución principal.

Capa de Comunicación y API: Para la interconexión entre el procesador local y el servidor de datos, se ha programado una API REST en PHP. Esta capa actúa como un cortafuegos lógico, encargándose de:

Criptografía: Procesamiento de contraseñas mediante el algoritmo de hash SHA-256, garantizando que ninguna clave se valide o compare en texto plano.

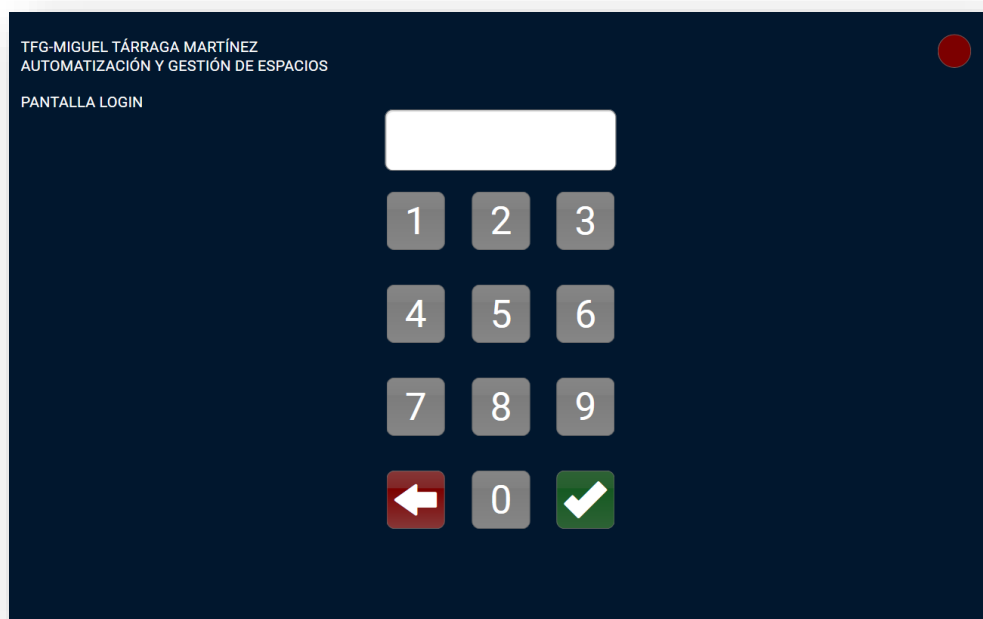
Prevención de Colisiones: Algoritmos matemáticos que evalúan las fechas de inicio y fin de las reservas para denegar inserciones que provoquen solapamientos de horario en la sala.

Operaciones CRUD: Controladores centralizados para la creación, lectura, actualización y eliminación de usuarios y eventos.

2. Entorno de ejecución.

a. Ejemplos y posibilidades.

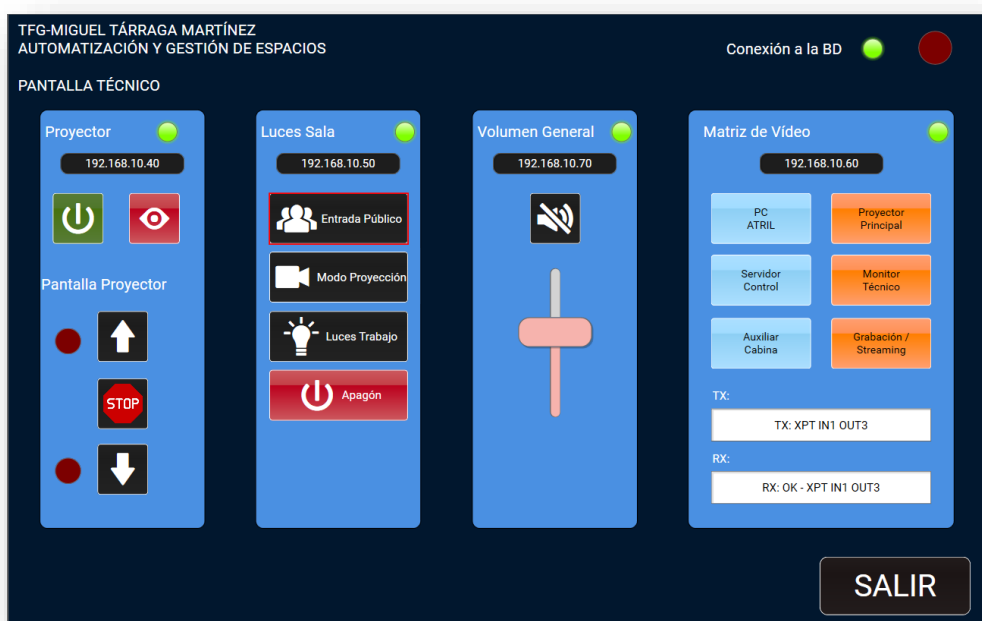
A continuación, se muestran capturas del prototipo de la Interfaz de Usuario (UCI) ejecutándose en el emulador, orientadas a su futura visualización en pantallas táctiles capacitivas QSC:



Captura 4. La pantalla de Login con el teclado numérico.



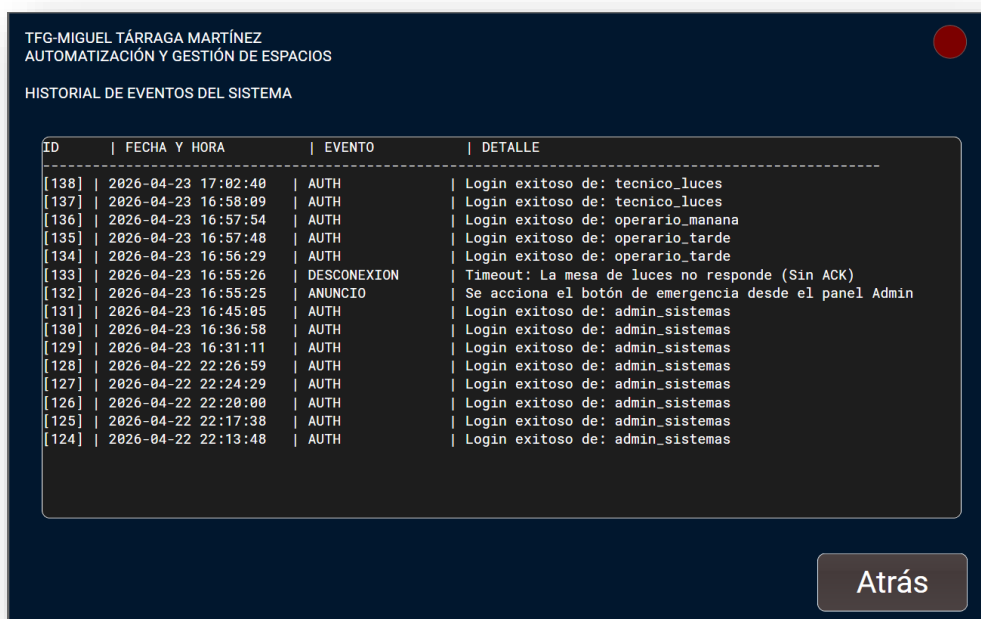
Captura 5. El Panel del Operario, mostrando los botones grandes de las Macros (Cine, Limpieza, etc.).



Captura 6. El Panel del Técnico, donde se ven los faders de Meyer, los botones de Evertz y los LEDs de conexión en verde/rojo.



Captura 7. El Dashboard del Administrador.



Captura 8. El Dashboard del Administrador mostrando el historial de eventos.

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA RESERVAS

ID

FECHA INICIO

FECHA FIN

DESCRIPCIÓN

Formato fecha: AAAA-MM-DD HH:MM

✓

↺

✖

PRÓXIMAS RESERVAS DEL TEATRO

[ID: 13]	2026-04-22 18:30 a 2026-04-22 20:30	Ensayo danza
[ID: 14]	2026-04-23 10:00 a 2026-04-23 14:00	Conferencia Ciberseguridad
[ID: 15]	2026-04-30 16:00 a 2026-04-30 21:00	Montaje Escenografía Obra Hamlet

Atrás

Captura 9. El Dashboard del Administrador mostrando las reservas.

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA ADMINISTRACIÓN DE USUARIOS

CREAR , MODIFICAR O ELIMINAR USUARIOS:

NOMBRE

PIN

ROL

CREAR

MODIFICAR

ELIMINAR

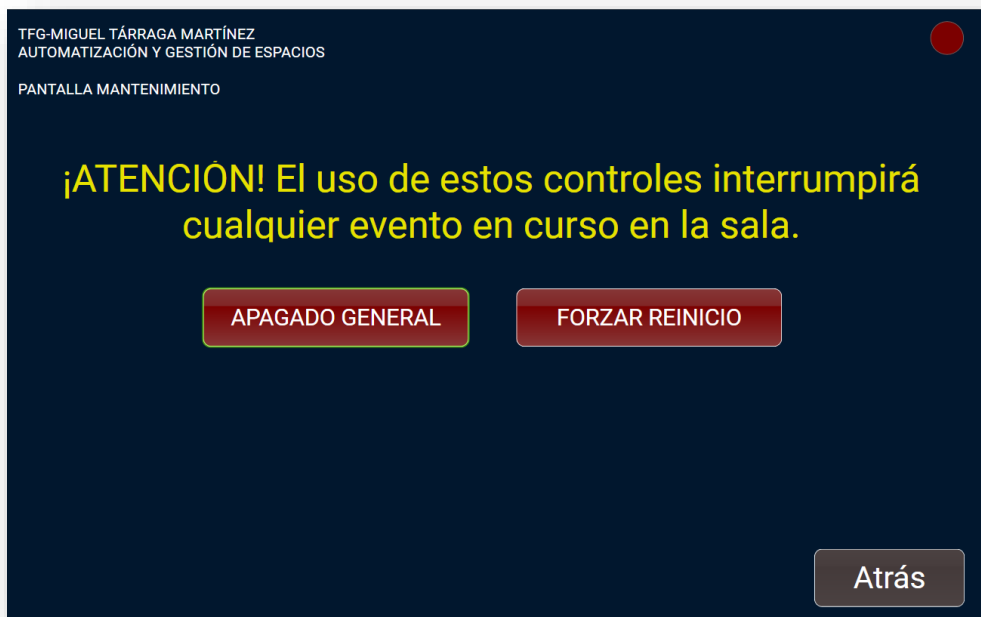
✓

↺

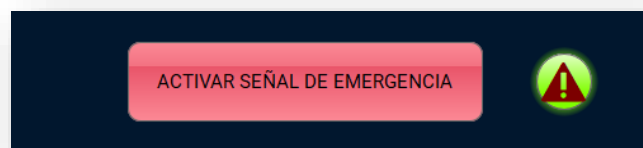
✖

Atrás

Captura 10. El Dashboard del Administrador mostrando gestión de usuarios.



Captura 11. El Dashboard del Administrador mostrando la ventana de mantenimiento.



Captura 12. El Dashboard del Administrador pulsando la señal de emergencia.

b. Tests.

Dado que el proyecto se ha validado sobre un entorno simulado, la batería de pruebas se ha centrado estrictamente en la auditoría por software, superando con éxito los siguientes escenarios de control de calidad:

- **Pruebas Unitarias:** Se ha validado la correcta transición dinámica de vistas basada en el Control de Acceso por Roles, demostrando que un Operario no puede acceder a las herramientas de red del Técnico ni a la base de datos del Administrador.
- **Pruebas de Integración:** Se ha comprobado la correcta lectura, escritura y persistencia de datos del código Lua contra la API de PHP. Se realizaron pruebas de estrés intentando crear reservas simultáneas o solapadas, confirmando que el *middleware* bloquea la petición y devuelve el error a la interfaz gráfica.
- **Pruebas de Tolerancia a Fallos:** Se han forzado caídas de red simuladas para evaluar la respuesta del código de control de hardware. Los temporizadores de *Watchdog* actúan según lo previsto, detectando la ausencia de respuesta del hardware y registrando el evento como una desconexión crítica en la tabla de logs de MySQL.

6. Prueba.

i. Elaborar pruebas y/o cuestionarios para detectar la utilidad, la valoración y los posibles errores.

Para estructurar la evaluación, se ha diseñado una serie de pruebas basada en roles y un cuestionario de valoración final.

1. Matriz de Tareas: Se asignan casuísticas específicas según el nivel de privilegio para evaluar si el usuario puede completarlas sin asistencia:

- **Rol Operario:** Iniciar sesión, ejecutar la macro "Modo Ensayo" y activar la "Señal de Emergencia".
- **Rol Técnico:** Silenciar el DSP de audio, enrutar la Entrada 2 a la Salida 1 en la matriz de vídeo y provocar un fallo de red artificial para comprobar si el sistema notificaba la avería.
- **Rol Administrador:** Registrar un nuevo usuario Operario en la base de datos, crear una reserva para el día siguiente y visualizar el histórico de logs.

2. Cuestionario de Usabilidad:

Tras la prueba, se aplicó un cuestionario basado en la escala Likert (del 1 al 5, donde 1 es "Muy en desacuerdo" y 5 es "Muy de acuerdo"), evaluando cuatro dimensiones críticas:

Intuitividad: "¿La interfaz deja claro qué botones pulsar sin leer un manual?"

Respuesta del sistema: "¿El programa informa correctamente de si una acción está cargando, ha tenido éxito o ha fallado?"

Prevención de errores: "¿El sistema te ha impedido realizar acciones destructivas o ilógicas (ej. solapar horas)?"

Sensación de seguridad: "¿Confías en que el sistema cierra tu sesión si te ausentas?"

ii. Procedimiento para la participación en la evaluación de los usuarios o clientes finales.

Al tratarse de un entorno de simulación local, el procedimiento de evaluación se realizó en un entorno controlado bajo las siguientes directrices:

- **Selección del grupo de control:** Se seleccionó a un grupo reducido de evaluadores (perfiles técnicos y no técnicos, simulando el personal real del teatro).
- **Entorno de prueba:** Los usuarios interactuaron con el emulador de Q-SYS Designer ejecutado a pantalla completa, conectado a la base de datos en local.
- **Ejecución ciega:** Se proporcionó a los usuarios únicamente sus credenciales de acceso, sin ofrecerles un manual previo. El objetivo era evaluar la curva de aprendizaje natural de la interfaz y la claridad de los mensajes de alerta.

Tras finalizar la lista de tareas, se procedió a cumplimentar el cuestionario y a una breve entrevista abierta para recoger impresiones cualitativas.

iii. Resultados obtenidos.

Los resultados de las Pruebas de Aceptación fueron altamente satisfactorios, validando la robustez del Producto Mínimo Viable y su capacidad para operar en un entorno real. De la evaluación se extrajeron las siguientes conclusiones principales:

- **Tasa de Éxito Funcional:** El 100% de las tareas críticas (control de bases de datos, ejecución de macros y ruteo técnico) fueron completadas con éxito. Los sistemas de seguridad respondieron correctamente ante intentos de uso indebido o caídas de red simuladas. Para dotar de veracidad empírica a estos resultados, se extraen las siguientes evidencias del sistema durante las pruebas:
 - **Evidencia 1 - Evaluación del Rol Operario:** El evaluador logró iniciar sesión, ejecutar la macro de automatización y accionar la alarma prioritaria sin asistencia. En la siguiente captura del historial de la base de datos se observa la correcta trazabilidad de estas acciones sucesivas (ejecución de escena y disparo de emergencia nivel 3) sin colapsos lógicos en el sistema.



Captura 13. El Dashboard del Operador pulsando el Modo Cine.

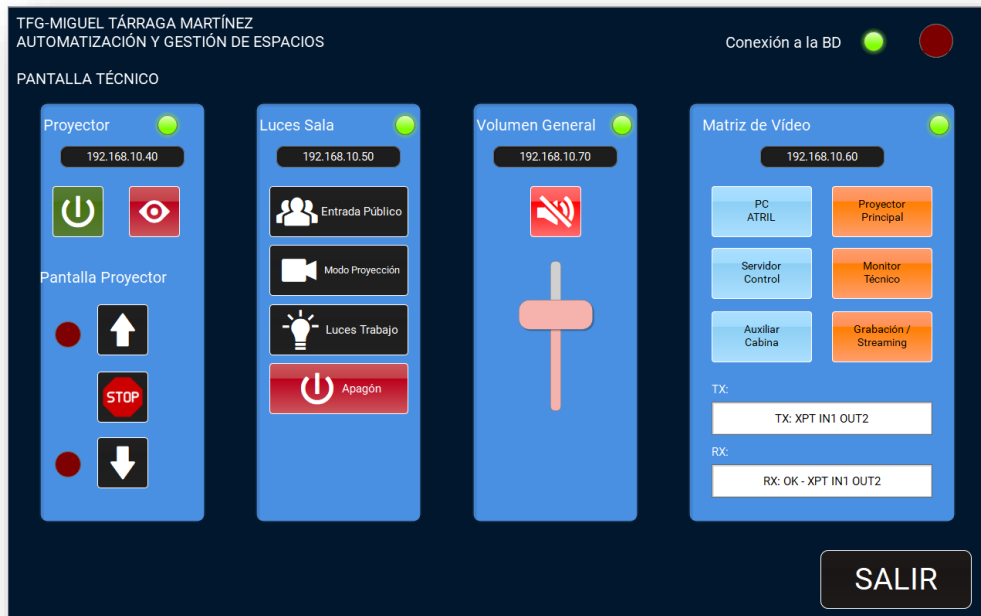


Captura 14. El Dashboard del Operador después de pulsar el botón de emergencia.

2026-05-06T16:17:03.370	LOG DB OK: ESCENA -> Activación de Modo Cine: Proyector ON, Luces Show, Audio -15dB
2026-05-06T16:17:16.840	MACRO: Timer reseteado a 60s
2026-05-06T16:17:16.840	ALERTA: Escucha las indicaciones.
2026-05-06T16:17:16.850	LOG DB OK: ANUNCIO -> Se acciona el botón de emergencia desde el panel Admin
2026-05-06T16:19:30.759	MACRO: Timer reseteado a 60s
2026-05-06T16:19:30.759	MACRO: Sistema ocupado, ignorando botón.
2026-05-06T16:19:33.645	MACRO: Timer reseteado a 60s
2026-05-06T16:19:33.645	MACRO: Sistema ocupado, ignorando botón.
2026-05-06T16:19:46.649	MACRO: Timer reseteado a 60s
2026-05-06T16:19:46.649	ALERTA: Anuncio finalizado. Sistema listo.
2026-05-06T16:19:57.505	MACRO: Timer reseteado a 60s
2026-05-06T16:19:57.505	MACRO: Ejecutando Escena -> CINEMA
2026-05-06T16:19:57.505	-> INFO: Bajando cortina, encendiendo luces show, volumen a -15dB, encendiendo proyector y ruteando Servidor a Proyector.
2026-05-06T16:19:57.549	LOG DB OK: ESCENA -> Activación de Modo Cine: Proyector ON, Luces Show, Audio -15dB

Captura 15. La ventana Debug, donde se puede apreciar todo el flujo realizado en esta tarea.

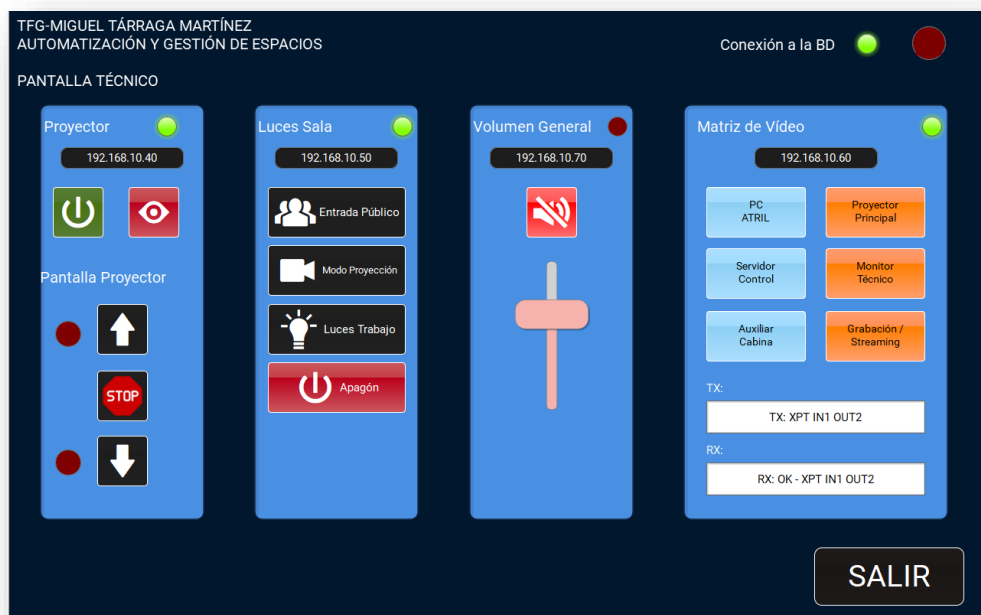
- **Evidencia 2 - Evaluación del Rol Técnico:** El evaluador completó el ruteo de señales y el silenciado de la sala. Para probar la lógica defensiva, se forzó la desconexión física de un equipo (simulando un fallo de red). Como demuestra la captura inferior, el Watchdog de Q-SYS detectó la ausencia de respuesta (paquete ACK) e insertó automáticamente la advertencia de Timeout en la base de datos, protegiendo la interfaz de bloqueos.



Captura 16. El Dashboard del Técnico silenciando el controlador de audio y enrutando la matriz de vídeo.

2026-05-06T16:32:52.192	Starting Script
2026-05-06T16:32:52.192	GALILEO FÍSICO: Esperando comandos en puerto 17000...
2026-05-06T16:32:52.192	Starting Script
2026-05-06T16:32:52.192	GALILEO FÍSICO: Esperando comandos en puerto 17000...
2026-05-06T16:32:52.688	GALILEO FÍSICO: Nuevo cliente conectado.
2026-05-06T16:33:09.078	GALILEO FÍSICO: Comando recibido -> SET MUTE ON
2026-05-06T16:41:06.887	Starting Script

Captura 17. La ventana Debug del simulador de audio, donde se puede apreciar que se ha presionado el botón "Mute ON".

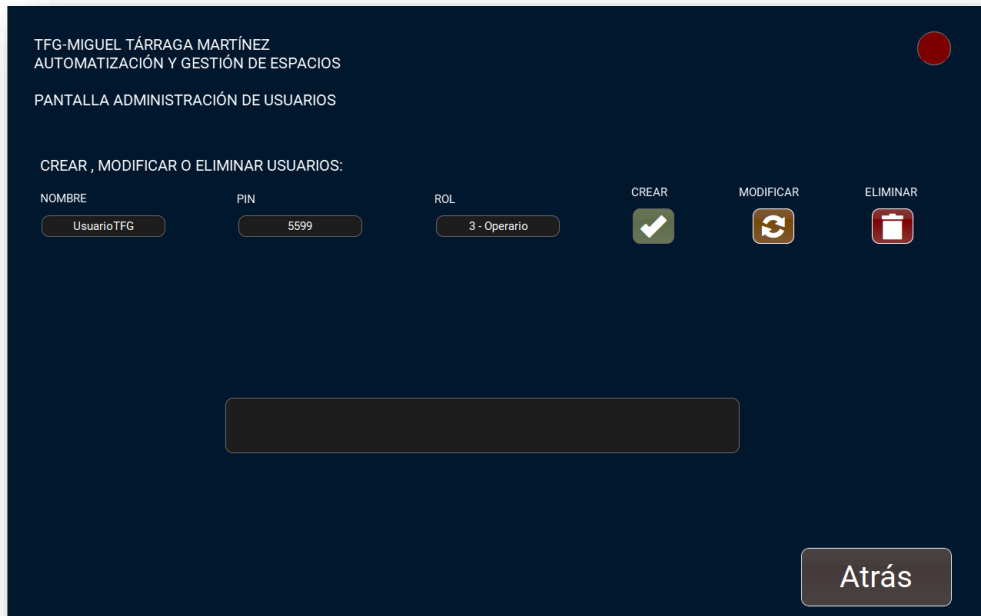


Captura 18. El dashboard del Técnico forzando la desconexión del equipo de audio (Se puede apreciar que el led permanece en color rojo).

2026-05-06T15:26:54.830	Starting Script
2026-05-06T15:26:55.338	AUDIO: Conectado a Galileo
2026-05-06T16:32:52.180	Starting Script
2026-05-06T16:32:52.180	Starting Script
2026-05-06T16:32:52.688	AUDIO: Conectado a Galileo
2026-05-06T16:41:06.890	AUDIO: Desconectado. Registrando avería en DB...
2026-05-06T16:49:10.628	AUDIO: Conectado a Galileo

Captura 19. La ventana Debug del Audio Controller, donde se puede apreciar que el procesador de audio se ha desconectado, registra la avería en la base de datos y al corregir la avería se vuelve a conectar.

- **Evidencia 3 - Evaluación del Rol Administrador:** El evaluador completó satisfactoriamente la inserción de un nuevo operario y el bloqueo horario de la sala. El algoritmo de la PHP validó las credenciales y demostró su eficacia al denegar un intento de registro de reserva que solapaba en horario con otra existente, devolviendo el correspondiente error por pantalla.



TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

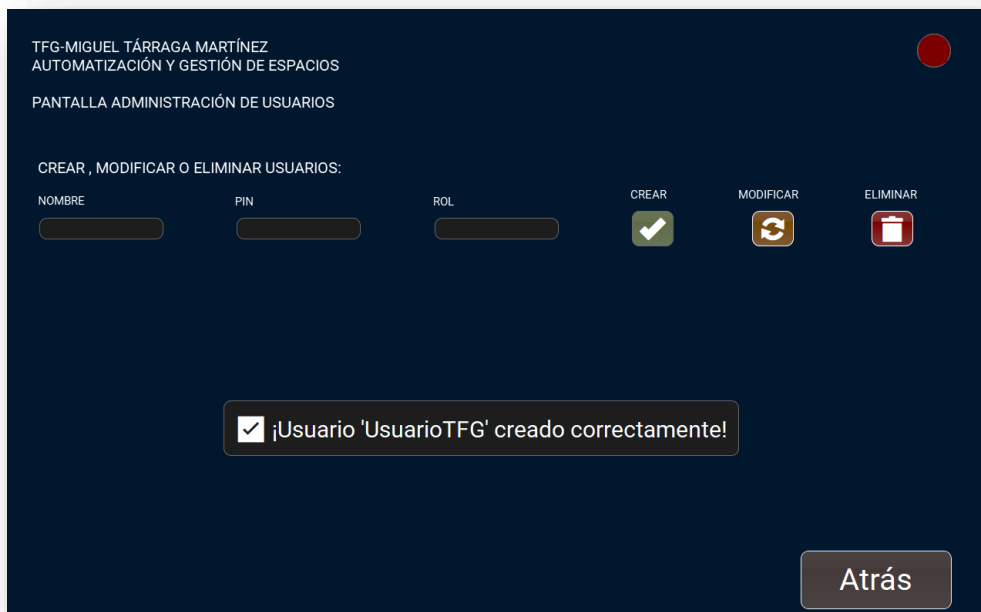
PANTALLA ADMINISTRACIÓN DE USUARIOS

CREAR, MODIFICAR O ELIMINAR USUARIOS:

NOMBRE	PIN	ROL	CREAR	MODIFICAR	ELIMINAR
UsuarioTFG	5599	3 - Operario			

Atrás

Captura 20. El dashboard del Administrador en la pantalla de administración de usuarios.



TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA ADMINISTRACIÓN DE USUARIOS

CREAR, MODIFICAR O ELIMINAR USUARIOS:

NOMBRE	PIN	ROL	CREAR	MODIFICAR	ELIMINAR

☒ ¡Usuario 'UsuarioTFG' creado correctamente!

Atrás

Captura 21. El dashboard del Administrador confirmando el usuario creado.

SELECT * FROM `usuario`

☐ Perfilando [\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Ordenar según la clave: Ninguna

Opciones extra

		id_usuario	nombre_usuario	clave_hash	id_rol	fecha_creacion
☐	Editar Copiar Borrar	1	admin_principal	03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e...	1	2026-03-26 20:07:30
☐	Editar Copiar Borrar	2	admin_sistemas	888df25ae3577242a560c7152a1de794440e0ea5cfee62828...	1	2026-03-26 20:07:30
☐	Editar Copiar Borrar	3	tecnico_av	f8638b979b2f4f793ddb6dbd197e0ee25a7a6ea32b0ae22f5e...	2	2026-03-26 20:07:30
☐	Editar Copiar Borrar	4	tecnico_luces	fe2592b42a727e977f055947385b709cc82b16b9a87f88c6ab...	2	2026-03-26 20:07:30
☐	Editar Copiar Borrar	5	operario_manana	0ffe1abd1a08215353c233d6e009613e95eec4253832a761af...	3	2026-03-26 20:07:30
☐	Editar Copiar Borrar	6	operario_tarde	edee29f882543b956620b26d0ee0e7e950399b1c4222f5de05...	3	2026-03-26 20:07:30
☐	Editar Copiar Borrar	24	userTest	03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e...	3	2026-05-06 20:41:48
☐	Editar Copiar Borrar	26	UsuarioTFG	a3f0fbc167fd10964710c3140c056bf91b2a27403ce706f74b...	3	2026-05-06 21:05:00

Captura 22. Servidor de la base de datos con el usuario creado

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA RESERVAS

ID	FECHA INICIO	FECHA FIN	DESCRIPCIÓN
	2026-05-07 10:00	2026-05-07 09:30	EnsayoTFG_HorarioMal

Formato fecha: AAAA-MM-DD HH:MM

☒ ☐ ☐

ERROR: Inicio debe ser anterior al fin.

PRÓXIMAS RESERVAS DEL TEATRO

```

[ID: 13] 2026-04-22 18:30 a 2026-04-22 20:30 | Ensayo danza
[ID: 14] 2026-04-23 10:00 a 2026-04-23 14:00 | Conferencia Ciberseguridad
[ID: 15] 2026-04-30 16:00 a 2026-04-30 21:00 | Montaje Escenografía Obra Hamlet
  
```

Atrás

Captura 23. El dashboard del Administrador creando una reserva con la fecha de fin anterior a la de inicio.

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA RESERVAS

ID	FECHA INICIO	FECHA FIN	DESCRIPCIÓN
	2026-04-30 13:00	2026-04-30 21:00	EnsayoTFG_HorarioSolapado

Formato fecha: AAAA-MM-DD HH:MM

☒ ☐ ☐

✖ ERROR: El horario ya está reservado.

PRÓXIMAS RESERVAS DEL TEATRO

```

[ID: 13] 2026-04-22 18:30 a 2026-04-22 20:30 | Ensayo danza
[ID: 14] 2026-04-23 10:00 a 2026-04-23 14:00 | Conferencia Ciberseguridad
[ID: 15] 2026-04-30 16:00 a 2026-04-30 21:00 | Montaje Escenografía Obra Hamlet
  
```

Atrás

Captura 24. El dashboard del Administrador creando una reserva con la fecha de otra reserva existente.

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA RESERVAS

ID	FECHA INICIO	FECHA FIN	DESCRIPCIÓN
	2026-05-07 10:30	2026-05-07 11:30	EnsayoTFG

Formato fecha: AAAA-MM-DD HH:MM

☒ ☐ ☐

PRÓXIMAS RESERVAS DEL TEATRO

```

[ID: 13] 2026-04-22 18:30 a 2026-04-22 20:30 | Ensayo danza
[ID: 14] 2026-04-23 10:00 a 2026-04-23 14:00 | Conferencia Ciberseguridad
[ID: 15] 2026-04-30 16:00 a 2026-04-30 21:00 | Montaje Escenografía Obra Hamlet
  
```

Atrás

Captura 25. El dashboard del Administrador creando una reserva (Antes de darle al botón insertar).

TFG-MIGUEL TÁRRAGA MARTÍNEZ
AUTOMATIZACIÓN Y GESTIÓN DE ESPACIOS

PANTALLA RESERVAS

ID: FECHA INICIO: FECHA FIN: DESCRIPCIÓN:

Formato fecha: AAAA-MM-DD HH:MM

☒ ☐ ☐

¡Reserva guardada correctamente!

PRÓXIMAS RESERVAS DEL TEATRO

[ID: 13]	2026-04-22 18:30 a 2026-04-22 20:30	Ensayo danza
[ID: 14]	2026-04-23 10:00 a 2026-04-23 14:00	Conferencia Ciberseguridad
[ID: 15]	2026-04-30 16:00 a 2026-04-30 21:00	Montaje Escenografía Obra Hamlet
[ID: 62]	2026-05-07 10:30 a 2026-05-07 11:30	EnsayoTFG

Atrás

Captura 26. El dashboard del Administrador con la reserva confirmada e insertada en la base de datos.

- **Valoración de Usabilidad (UX):** La interfaz fue calificada como "muy intuitiva". Los usuarios destacaron positivamente el *feedback* visual (mensajes de estado y LEDs), lo que redujo la curva de aprendizaje y la incertidumbre operativa durante las maniobras críticas.
- **Identificación de puntos de mejora:** Si bien la valoración general fue excelente, la observación directa y el *feedback* de los evaluadores permitieron identificar **dos áreas de mejora prioritarias** que afectaban a la fluidez del trabajo diario:
 - Se detectó que el tiempo de sesión era demasiado restrictivo para perfiles administrativos.
 - Se identificó un riesgo de duplicidad de datos en los formularios por la persistencia de texto tras la ejecución de comandos.

Estos hallazgos fueron procesados de inmediato, dando lugar al replanteamiento técnico que se detalla en el siguiente apartado de Mantenimiento.

7. Mantenimiento

El diseño modular del software permite que el ciclo de vida de la aplicación se adapte tanto a los errores detectados en fase de pruebas como a las futuras necesidades del entorno real.

7.a. Replanteamiento del proyecto según las pruebas realizadas

Tras el análisis de las Pruebas de Aceptación de Usuario detalladas en el apartado anterior, se han replanteado y modificado diversos pasos de la implementación original para optimizar el producto final:

- **Ajuste de la Lógica de Sesión:** El diseño inicial contemplaba un cierre de sesión a los 30 segundos, pero las pruebas de campo demostraron que era ineficiente operativamente. Se replanteó el script `Login_Controller` para ampliar la tolerancia a 60 segundos, logrando un equilibrio entre seguridad y usabilidad.
- **Replanteamiento de la Interfaz y Limpieza de Variables:** Las pruebas revelaron envíos duplicados por retención de datos en memoria. Esto obligó a rediseñar el flujo de los formularios en Q-SYS, añadiendo funciones de limpieza (`limpiarFormularioUsuarios` y `limpiarFormularioReservas`) tras cada ejecución de la API REST.

7.b. Mantenimiento Preventivo y Correctivo

Para garantizar la operatividad del sistema, se establecen las siguientes rutinas:

- **Auditoría mediante Logs:** El uso de la tabla “Log_Evento” permite un mantenimiento correctivo ágil. El personal técnico puede identificar si un fallo es de origen físico o lógico sin necesidad de intervenir físicamente en los equipos.
- **Gestión de Base de Datos:** Se recomienda la ejecución periódica de copias de seguridad del entorno mediante el panel de administración de XAMPP.

7.c. Escalabilidad y Expansión del Sistema

La arquitectura de tres capas (Q-SYS, PHP, MySQL) confiere al proyecto una gran capacidad de crecimiento:

- **Escalabilidad de Hardware:** Añadir nuevos dispositivos solo requiere instanciar un nuevo bloque de script en Q-SYS Designer y enlazar su señal de latido al controlador de seguridad existente, sin alterar el núcleo del programa.
- **Escalabilidad de Software:** El uso de una API REST descentralizada permite que, en el futuro, otros dispositivos (como una aplicación móvil externa para el personal del barco) puedan consumir la misma base de datos para consultar el estado de la sala o el calendario de reservas de forma remota.

7.d. Actualizaciones de Entorno

Para garantizar la seguridad perimetral y la estabilidad del código a largo plazo, el plan de mantenimiento contempla:

- **Actualizaciones del Firmware Q-SYS:** Al utilizar código Lua nativo y funciones estándar de red, el diseño es altamente compatible con futuras actualizaciones del sistema operativo del Core 24f, minimizando el riesgo de funciones obsoletas.
- **Parches de Seguridad Web:** Mantener el entorno del servidor (Apache/PHP/MySQL) actualizado a sus últimas versiones estables para asegurar que los algoritmos de cifrado y la protección contra inyecciones SQL se mantengan invulnerables frente a nuevas amenazas de ciberseguridad.

8. Conclusiones.

El desarrollo de este Trabajo de Fin de Grado ha supuesto un desafío técnico integral, marcando la transición desde los conocimientos teóricos adquiridos durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma y las certificaciones oficiales obtenidas en el ecosistema Q-SYS, hacia la implementación de una solución de software de grado industrial.

El proyecto ha permitido fusionar dos mundos tradicionalmente separados: el desarrollo de software puro y la ingeniería audiovisual. Tras la finalización de las fases de desarrollo y la ejecución de las pruebas, se extraen las siguientes conclusiones:

8.a. Consecución de los objetivos planteados

Se ha logrado diseñar, programar y validar con éxito un Producto Mínimo Viable (MVP) que unifica el control de hardware físico y la gestión de datos en una única plataforma. La integración de la capa de presentación y lógica local con el *middleware* y el *backend* ha demostrado ser una arquitectura altamente eficiente. El sistema cumple estrictamente con los requisitos del entorno marítimo/teatral para el que fue diseñado, ofreciendo seguridad, automatización y telemetría en tiempo real.

8.b. Dificultades técnicas superadas

Durante el ciclo de desarrollo, se presentaron diversos obstáculos técnicos que fueron resueltos aplicando patrones de diseño avanzados:

Comunicaciones Asíncronas y Sockets de Red: Gestionar la comunicación bidireccional mediante protocolos TCP y UDP sin bloquear el hilo principal de la interfaz fue un reto que se resolvió mediante el uso de temporizadores y rutinas *Watchdog*.

Sincronización de Estados y Seguridad: Lograr que múltiples controladores de hardware independientes notificaran su actividad a un controlador central para mantener la sesión viva fortaleció la seguridad del sistema contra accesos no autorizados.

Integridad Lógica en la Base de Datos: La creación del motor de reservas exigió desarrollar algoritmos en el servidor capaces de prevenir colisiones horarias en tiempo real, garantizando la integridad de negocio de la base de datos.

8.c. Crecimiento personal y profesional

A nivel formativo, este proyecto ha supuesto una especialización única. He logrado hibridar las competencias de desarrollo multiplataforma de DAM con los estándares de control profesional avalados por mis certificaciones oficiales en Q-SYS.

Esta combinación me ha permitido consolidar conocimientos críticos en áreas donde el software interactúa con el mundo físico:

Especialización en Control AV: Aplicación práctica de las certificaciones de QSC para el manejo de protocolos industriales y el motor de *scripting* avanzado.

Seguridad Criptográfica: Uso de algoritmos de *hash* (SHA-256) para la protección de credenciales.

Arquitecturas Limpias: Separación de responsabilidades mediante la creación de una API REST que actúa como escudo entre la interfaz táctil y el servidor de datos.

Lógica Defensiva: Diseño de interfaces restrictivas basadas en roles que impiden errores operativos y protegen la integridad del hardware.

8.d. Valoración final

Más allá del ámbito estrictamente académico, este proyecto representa un punto de inflexión clave en mi desarrollo profesional. Actualmente, desempeño mi labor en una empresa del sector de las instalaciones y proyectos audiovisuales, donde hasta ahora mi perfil ha estado enfocado en la parte física (técnico de cableado estructurado e integración en racks).

El desarrollo de este Trabajo de Fin de Grado me ha proporcionado las herramientas y la confianza necesarias para dar el salto hacia la ingeniería de control y la programación de sistemas AV. La perspectiva de futuro inmediata para este proyecto es su aplicabilidad real: el objetivo es utilizar este Producto Mínimo Viable (MVP) como núcleo base, sacarlo del entorno del emulador y volcarlo en los procesadores físicos de la empresa. Las siguientes fases orgánicas consistirán en adaptar esta lógica a las necesidades exactas exigidas por la sala real del buque, realizar la interconexión con los dispositivos físicos definitivos y poner el sistema en producción.

En conclusión, el ecosistema de software desarrollado no solo certifica la asimilación completa de las competencias del título de DAM, sino que se ha convertido en una herramienta tangible que impulsará mi transición profesional de técnico instalador a desarrollador e integrador de sistemas multiplataforma en el sector audiovisual.

9. Bibliografía y/o webgrafía

Para la planificación, desarrollo y ejecución de este Trabajo de Fin de Grado, se ha consultado la documentación oficial de los lenguajes de programación empleados, así como los manuales de integración de los distintos fabricantes de hardware audiovisual.

9.a. Desarrollo de Software y Entorno Q-SYS

- **QSC Audio Products.** *Q-SYS Designer Software Help File*. Manual oficial de referencia para el desarrollo de la Interfaz de Usuario (UI), uso del entorno de emulación y configuración de pines lógicos. Disponible en: <https://gsyshelp.qsc.com/>
- **Q-SYS Developer Community.** Repositorios y foros oficiales de integración para programadores de la plataforma. Disponible en: <https://developers.qsc.com/>
- **Ierusalimschy, R., de Figueiredo, L. H., & Celes, W.** *Lua 5.3 Reference Manual*. Documentación oficial del lenguaje utilizado para la programación asíncrona, temporizadores y parseo de datos en el *Scripting Engine* de Q-SYS. Disponible en: <https://www.lua.org/manual/5.3/>
- **Tencent.** *RapidJSON Documentation*. Librería empleada dentro del código Lua para la codificación y decodificación eficiente de paquetes JSON entre el cliente y el servidor. Disponible en: <https://rapidjson.org/>

9.b. Middleware, Backend y Bases de Datos

- **The PHP Group.** *PHP Manual*. Documentación técnica empleada para el desarrollo de la API REST, la gestión asíncrona de peticiones HTTP, y la implementación del algoritmo criptográfico SHA-256. Disponible en: <https://www.php.net/manual/es/>
- **Oracle Corporation.** *MySQL 8.0 Reference Manual*. Manual de referencia para el diseño de la base de datos relacional, normalización de tablas (usuarios, reservas, logs) y ejecución segura de operaciones CRUD. Disponible en: <https://dev.mysql.com/doc/refman/8.0/en/>
- **Apache Software Foundation.** *Apache HTTP Server Documentation*. Base técnica para el despliegue del entorno servidor local (XAMPP). Disponible en: <https://httpd.apache.org/docs/>

9.c. Protocolos de Red e Integración de Hardware AV

- **Meyer Sound Laboratories.** Especificaciones de telemetría y comandos de control sobre protocolos TCP/IP y SNMP para el procesador de audio Galileo Galaxy.
- **ETC.** *EOS Family Show Control User Guide*. Especificaciones para la integración y disparo de *Cues* mediante el protocolo UDP y tramas OSC (Open Sound Control).
- **Christie Digital Systems.** *Serial Commands and API*. Documentación para el control de encendido, apagado y obturador de lente de sistemas de videoproyección.
- **Evertz Microsystems.** *Magnum / Matrix CLI Reference Guide*. Manual de comandos Telnet (Puerto 23) para el enrutamiento de señales de vídeo.

9.d. Repositorio del Proyecto y Código Fuente

- **Tárraga Martínez, Miguel.** *TFG - Automatización y gestión de espacios*. Repositorio oficial en GitHub que contiene la arquitectura completa del proyecto: controladores de Q-SYS (Lua), scripts de integración PHP y el esquema de base de datos MySQL.

Enlace al proyecto:

https://github.com/mtarraga/TFG-Automatizacion_y_gestion_de_espacios